AIOA spArkHAT · ARCHITECTURE & EPISTEMIC AUDIT

GMAIL API GATEWAY

Pięć audytów AI. Jedna decyzja architektoniczna.

Meta · DeepSeek · Claude Sonnet · Gemini · Kimi

20 października 2026 | Raport projektowy, archiwum odpowiedzi i plan wdrożenia

> ***Status:*** *Wybór architektury i archiwizacja audytów zakończone. Kod Gmail Gateway nie został jeszcze zaimplementowany. Nie podłączono konta Google, nie wykonywano OAuth ani nie wysyłano poczty. Cały dokument jest materiałem do zatwierdzenia przed kodowaniem.*

Od idei AIOA do pięciu niezależnych audytów

Od początku AIOA spArkHAT rozwijamy według jednej zasady: **model może doradzić, ale nie staje się autorytetem wykonawczym**. W kolejnych etapach projektu powstały mechanizmy Non-Zero, ServiceGuard, dowody pochodzenia, krytyka modeli, HAT, checkpointy i kontrola człowieka. Zamiast dopinać Gmail „na skróty", zrobiliśmy to, co pasuje do tej filozofii: zapytaliśmy **pięć ulubionych modeli** o niezależny audyt architektury i ryzyk.

Poprosiliśmy Meta AI, DeepSeek, Claude Sonnet, Gemini i Kimi o projekt połączenia Gmail API z AIOA. Każdy mógł zakwestionować nasze założenia. Wyniki nie są pięcioma głosami decydującymi o prawdzie; to materiał porównawczy, z którego główny architekt wyprowadza jedną spójną wersję.

Niniejszy raport kończy fazę zbierania stanowisk i **przygotowuje start implementacji G0–G1 offline** po dalszej dyspozycji właściciela.

Dane i autentyczność archiwum

Numer	Model	Stan materiału	Co zawiera aneks
A01	Meta AI	Podsumowanie	Otrzymane stanowisko, nie pełny plik modelu
A02	DeepSeek	Długa odpowiedź (17 sekcji)	Obszerna ustrukturyzowana transkrypcja z archiwum; nie dosłowny eksport 1:1
A03	Claude Sonnet	Rozbudowane podsumowanie	Otrzymana odpowiedź; model oferował dopiero pełniejszy raport
A04	Gemini	Długa odpowiedź	Obszerna ustrukturyzowana transkrypcja z archiwum; zachowane rozbieżności

A05	Kimi	Podsumowanie	Główne ustalenia; pełny raport nie został dostarczony

Oryginalne wiadomości od użytkownika pozostają w rozmowie ChatGPT. Archiwum GitHub rejestruje otrzymane treści i ich uporządkowane opracowania. W aneksach **nie poprawiamy wypowiedzi modeli przez ciche usuwanie błędów**; korekty znajdują się w oddzielnym rozdziale.

CZĘŚĆ I — DECYZJA GŁÓWNEGO ARCHITEKTA

1. Rozstrzygnięcie ADR-GMAIL-001

Wybór: mały, **modułowy adapter Gmail w Pythonie**, początkowo **wewnątrz istniejącego runtime AIOA**, za pojedynczym Core i oryginalnymi bramkami ServiceGuard/Non-Zero. Oddzielamy poufne poświadczenia przez lokalny broker tokenów, ale nie budujemy na etapie MVP osobnego publicznego REST API ani drugiego procesu decyzyjnego.

Dlaczego: jest to najmniejsza powierzchnia integracji, szybciej ją przetestujemy na komputerze z około 4 GB RAM i łatwiej połączymy później z głównym kodem. Zasadę z Meta — silne oddzielenie poświadczeń — zachowujemy jako izolację uprawnień. Osobny proces sidecar można rozważyć po udowodnieniu, że realnie zwiększy ochronę i nie stworzy drugiej autoryzacji.

Jedna zasada: źródło wiadomości, model, Gmail API, telefon Android i przyszły Chromium nie mogą samodzielnie nadawać praw. AIOA Core jest jedyną ścieżką zgody na skutek zewnętrzny.

2. Schemat systemu

```
ANDROID / CHATGPT / MODELE
    |  draft, analiza, propozycja
    v
AIOA CORE (single authority)
    |  ServiceGuard / Non-Zero / exact owner approval
    |  STOP / scope / replay / checkpoint
    v
CommunicationPort -> MailProviderPort
    |
    +--> GmailAdapter --------------- HTTPS ---> GMAIL API
    |       |
    |       +--> OS Credential Broker (Linux Keyring)
    |       +--> Durable Outbox Ledger (UNKNOWN_SEND)
    |       +--> Receipt / audit / idempotency
    |
    +--> Reply Evidence -> sanitized, untrusted
                             |
                             v
                      HAT STAGING (review needed)

FUTURE: Managed Chromium adapter under the SAME Core gates
        (interface planning ONLY; no Chromium coding here)
```

Nie tworzymy oddzielnego systemu nadawania zgód w module Gmail. Każda wiadomość musi mieć właściciela, źródło, zadanie, ograniczenia, zakres uprawnień, czas ważności i dowód pojedynczej próby wysłania.

3. Wysyłanie e-maila: wymagana maszyna stanów

```
PROPOSED -> WAITING_OWNER -> APPROVED -> SENDING
                                      |         |
                                      |         +--> SENT_ACCEPTED_BY_API
                                      |         +--> UNKNOWN_SEND
                                      |
                                REJECTED / EXPIRED / STOPPED
```

- Model tworzy tylko **MessageProposal** — propozycję, nie rozkaz wykonania.
- Człowiek widzi **rzeczywistego odbiorcę, temat, treść, załączniki i zakres danych**; zatwierdza konkretną operację.
- Core wiąże zgodę z **kanonicznym hashem końcowych bajtów wiadomości MIME** oraz task_id, sender/To/Cc/Bcc, subject, thread, scope, nonce, expiry i attachments. Nie wystarczy skrót części pól.
- Ledger zapisuje **trwałą próbę przed kontaktem z siecią**. Po rozpoczęciu wysyłania token zgody nie może zostać użyty drugi raz.
- Gmail messages.send **nie zapewnia natywnego klucza idempotencji**. Awaria po żądaniu może oznaczać faktyczną wysyłkę, więc wynik przechodzi do **UNKNOWN_SEND**.
- Nigdy nie wysyłamy automatycznie ponownie po UNKNOWN. Odczyt Sent/Message-ID/Thread to pomoc przy rozpoznaniu, ale **brak wiadomości w wyszukiwaniu nie dowodzi niewysłania**.
- Użytkownik może po analizie podjąć **nową, osobną decyzję**, z wyraźnym ostrzeżeniem o ryzyku duplikatu.

Sukces API potwierdza przyjęcie żądania przez Google; **nie jest jeszcze dowodem doręczenia odbiorcy**.

4. Scopes, OAuth i polityka Google — weryfikacja źródłowa

Zakres	Aktualna klasyfikacja Google	Nasza decyzja
gmail.send	**Sensitive**	Pierwszy pilot: tak, tylko po właścicielskiej zgodzie
gmail.readonly	**Restricted**	Późniejszy, osobny etap czytania odpowiedzi
gmail.compose	**Restricted**	Niepotrzebny na MVP: draft zapisuje AIOA
gmail.metadata	**Restricted**	Bez potrzeby na start
gmail.modify	**Restricted**	Nie używać w pierwszym etapie
gmail.labels	**Non-sensitive**	Niepotrzebny do wysłania pytania
mail.google.com	**Restricted / szeroki dostęp**	Wyłączony przez naszą politykę

Uwaga: Gemini odwrócił klasyfikacje dla gmail.send i gmail.compose/metadata. Obowiązuje oficjalna tabela Google, nie opinia modelu.

OAuth: lokalny klient Desktop dla Linux Mint/Ubuntu, Authorization Code + PKCE S256, krótkotrwały listener tylko na 127.0.0.1, port efemeryczny, state przeciw CSRF, systemowa przeglądarka Linux. Przestarzały OOB/manual copy-paste nie jest ścieżką MVP. Refresh token w chronionym systemowym magazynie, nie w Git, logach, HAT ani na Androidzie. Samo chmod 0600 nie jest szyfrowaniem.

Dla zewnętrznej aplikacji w trybie Testing z Gmail scopes zazwyczaj występuje 7-dniowa ważność refresh tokenu. To ograniczenie pilotażowe, nie gwarancja przyszłej pracy produkcyjnej. Rzeczywista weryfikacja

aplikacji Google/CASA zależy od zakresów, udostępniania i sposobu przetwarzania danych — **nie traktujemy wycen i terminów modeli jako zatwierdzonych**.

Oficjalne źródła:

- https://developers.google.com/workspace/gmail/api/auth/scopes
- https://developers.google.com/identity/protocols/oauth2/native-app
- https://developers.google.com/identity/protocols/oauth2/
- https://developers.google.com/workspace/gmail/api/reference/rest/v1/users.messages/send
- https://developers.google.com/workspace/gmail/api/reference/quota
- https://developers.google.com/terms/api-services-user-data-policy

5. Scenariusz demonstracyjny: pytanie do Natalii

1. AIOA wykonuje zadanie i identyfikuje brak informacji.
2. Jeden model proponuje pytanie, drugi może skrytykować jego treść.
3. Core prezentuje właścicielowi pełny podgląd: odbiorca, pytanie, kontekst, potencjalne dane prywatne.
4. Człowiek zatwierdza konkretną wiadomość; brak zgody = brak wysyłki.
5. Gmail Gateway wykonuje najwyżej jedną próbę. Zachowuje bezpieczny receipt.
6. W wariancie send-only właściciel może dostarczyć odpowiedź ręcznie jako **niezweryfikowany input**.
7. Późniejszy moduł selected-thread READ doda odczyt wybranych odpowiedzi, dopiero po osobnej zgodzie i ocenie ograniczeń restricted scopes.
8. Odpowiedź jest **informacją / potencjalnym potwierdzeniem w granicach swojej treści**, nie pozwoleniem do instalacji, shell, GitHub push, wydatków, ani zmian polityk.
9. HAT przyjmuje tylko staged/untrusted evidence. Osobny audyt decyduje o promocji wiedzy.

Nigdy nie przypisujemy autorstwa lub zgody Natalii na podstawie samego From:, Message-ID czy DKIM; wysokie ryzyko wymaga dodatkowego potwierdzenia.

6. Decyzje o bezpieczeństwie

- **Credentials:** tylko Linux OS keyring / zatwierdzony broker, niewidoczne dla modeli, backendu testowego i Androida.
- **Minimum data:** brak kopiowania całej skrzynki; wybieramy pojedyncze wątki; brak pełnych wiadomości w GitHub/evidence/logach.
- **External models:** tekst maila nie trafia do Gemini/Claude/OpenRouter bez osobnej polityki udostępniania i zgody.
- **Attachments:** limit wielkości i typów, skanowanie/bezpieczny staging, bez wykonywania zawartości.
- **Prompt injection:** treść maila to obce dane, nigdy instrukcja.
- **STOP:** uniemożliwia następny start; nie cofa już wysłanego żądania, a operacja w locie pozostaje UNKNOWN.
- **Per-message approval:** wygaśnięcie, zakres, podpis/receipt oraz pojedyncze zużycie; zwykłe pole boolean approved_by_owner nie wystarcza.
- **Runtime:** osobny katalog roboczy dla Gmail Gateway, osobne dowody i limity; nie dotykamy kwalifikacji Linux S33/S34.
- **No public REST:** interfejs dla przyszłego Chromium to kontrakt portów, bez tworzenia publicznego endpointu.
- **GDPR/Limited Use:** projektujemy minimalizację, retencję/usunięcie i ostrożność wobec danych współpracowników.

7. Plan wykonawczy G0–G5

Faza	Co tworzymy	Kryterium przejścia
G0	Inspekcja aktualnego runtime; kontrakty MessageProposal/MailProviderPort, mapa authority, ADR i scenariusze negatywne	Przegląd źródeł, bez sieci, brak drugiego Core
G1	Fake Gmail API, Outbox Ledger, exact MIME hash, offline crash/replay/UNKNOWN + testy	PASS negatywnych przypadków i izolacja od Linux sprints
G2	Właścicielski OAuth Desktop, PKCE, keyring, dedykowany projekt GCP	Osobna zgoda, testowy Gmail, zero tokenów w logach
G3	Kontrolowana wysyłka jednej wiadomości **do siebie**	Jawne zatwierdzenie treści, recipient i najwyżej jeden send
G4	Opcjonalny selected-thread read, multi-model drafting/critic, HAT staging	Oddzielne scope/consent i privacy gate; zero auto-promotion
G5	Widok operatora Android pod tym samym Core	Brak drugiego signera i tokenów Gmail na telefonie

Szacunek: G0–G1 to zwykle kilka dni inżynierii zależnie od istniejących portów. Pierwsza próba rzeczywistej wysyłki może nastąpić po pomyślnym G0/G1/G2 i Twojej zgodzie; nie ma gwarantowanej daty. Dokończenie G4/G5 jest oddzielną pracą.

8. Integracja z GitHub, Linux i GOLD72

Repozytorium: https://github.com/luciferprosun/AIOA-spArkHAT **Dedykowana gałąź:** feature/aioa-gmail-gateway **Publiczna baza gałęzi:** 81656e6ccbb0c02307d049e865b6df3225ddb578 **Aktualna ścieżka dokumentacji audytowej:** docs/audits/gmail_gateway/

W tym czasie Linux Sprint 33/34 jest prowadzony niezależnie, a GOLD72 nie jest źródłem uprawnień dla Gmail. Pierwsze G0/G1 może działać na izolowanym worktree. Nie wolno zmieniać źródeł zapieczętowanych sprintów ani czasu testu 72h. Na merge z main czekamy do pełnej weryfikacji aktualnych różnic, regresji i zatwierdzenia właściciela.

Przyszłość: kanalizacja provider-neutral pozwoli w przyszłości dodać Chromium/Muse, ale obecnie NIE implementujemy Chromium ani przeglądarkowego Gmaila.

9. Co dalej po tym dokumencie?

Gdy właściciel da polecenie, rozpoczynamy **G0**: rozpoznanie portów AIOA, kontrakty Python, testy bezpieczeństwa i implementację offline na osobnym branchu. Nie łączymy jeszcze Google, nie wysyłamy e-maili i nie naruszamy Linux roadmapy.

> ***Ostateczna decyzja:*** *pięć modeli dostarczyło materiału audytowego. Wybieramy jedno Core, wąski adapter, ścisłe hash-bound approvals, trwały ledger, bezpieczną obsługę UNKNOWN_SEND i stopniowe włączanie Gmail API. Zaczynamy kodowanie od testów offline, nie od tokenów.*

CZĘŚĆ II — ANEKSY Z PIĘCIU AUDYTÓW

Poniżej znajdują się treści archiwizacyjne z naszego publicznego brancha GitHub. Zawierają wyłącznie to, co otrzymaliśmy od modeli lub co zostało wcześniej jawnie opisane jako uporządkowana transkrypcja. Stanowiska modeli nie są automatycznie przyjmowane jako fakty lub uprawnienia.

A01 — Meta AI — archiwum odpowiedzi

Źródło GitHub: https://github.com/luciferprosun/AIOA-spArkHAT/blob/feature/aioa-gmail-gateway/docs/audits/gmail_gateway/2026-10-20_META_AI_AUDIT_RECEIVED.md

---

AIOA Gmail Gateway — Meta AI independent audit (received summary)

- **Auditor (as identified by sender):** Meta AI
- **Received:** 2026-10-20
- **Project:** AIOA spArkHAT
- **Target branch:** `feature/aioa-gmail-gateway`
- **Evidence type:** User-pasted auditor summary, preserved without independent validation
- **Full report claimed by auditor:** `AIOA_Gmail_Gateway_Arch_Report.md`
- **Full report received as attachment: NO — NOT AVAILABLE**. This file does not represent or reconstruct the full report.
- **Review status:** `RECEIVED_SUMMARY_ONLY / INDEPENDENT_FACT_CHECK_PENDING / NOT_APPROVED_FOR_IMPLEMENTATION`
- **Secrets:** None intentionally included

Exact supplied response

> *Meta Full independent audit completed — research only, no code changes.*
>
> *I've synthesized all official Google policy findings, quota/idempotency behavior, and AIOA integration analysis into a professional architectural report:*
>
> 📄 📎 ***AIOA_Gmail_Gateway_Arch_Report.md***
>
> *TL;DR - Principal Architect Recommendation:*
>
> *Architecture: Sidecar Communication Gateway (not in-process library). AIOA Core remains sole authority, sidecar owns OAuthCredentialBroker + OutboxLedger, models never see tokens.*
>
> *Minimum safe scopes:*
>
> *MVP A (send only): gmail.send only — sensitive, not restricted vs restricted list, so no CASA blocking*
>
> *MVP B/C (read + monitor reply): Requires gmail.readonly (restricted) → triggers 6-week verification + annual CASA assessment $500-4500/yr*
>
> *Fastest to first real email:*
>
> *G0 contracts (2d) → G1 offline simulator with UNKNOWN handling (3d) → G2 PKCE loopback bootstrap (2d) → G3 one exact owner-approved send via users.messages.send, verify raw MIME base64url and no idempotency.*
>
> *Exact conditions to connect account: Linux Desktop OAuth client + loopback 127.0.0.1:random + PKCE S256, encrypted storage 0600, Testing mode limits (100 users, 7-day refresh expiry), owner runs bootstrap locally.*
>
> *Must remain blocked: All restricted scopes, auto-retry of UNKNOWN_SEND (requires manual Sent folder check), token on Android, unauthenticated REST for future Chromium, Gmail body in logs/GitHub, foundation model training per Limited Use, and any GOLD72 sealed source modification.*

> *The report includes: 2 architectures compared (tight vs sidecar) Full typed contracts (Pydantic) 14-scenario threat model with prevention/detection/recovery/tests G0-G5 roadmap with time estimates for 4GB Mint + Android operator GDPR Germany compliance mapping 10 top risks and unsafe assumptions in proposal*
>
> *All conclusions cite official docs — OAuth native flow, loopback migration, quotas, User Data Policy, and scope tables.*

Review/verification queue (not endorsement)

1. Verify official Google scope classifications and separate **restricted scope classification**, **OAuth verification requirements**, and **CASA assessment applicability**; their conditions are not universal simply because a scope is requested.
2. Check quoted cost and six-week lead time as **unverified model estimates**, not confirmed Google policy.
3. Compare sidecar process costs, IPC attack surface, token custody, restart/durability and resource isolation against the existing single-core AIOA architecture.
4. Check that `0600` file permissions and encryption claims are independently specified; permissions alone do not encrypt credentials.
5. Verify sender-message correlation cannot be equated with authentication of a person and that `UNKNOWN_SEND` never triggers blind resend.
6. Request the full `AIOA_Gmail_Gateway_Arch_Report.md` to inspect official URLs, threat matrix, contracts and actual recommendations before accepting findings.
7. Respect the user's boundaries: no Gmail OAuth connection, outbound email, Chromium implementation, LIVE activation, or changes to main based solely on this report.

Provenance notes

This document was transcribed from the user's 2026-10-20 chat message. The quoted claims are statements of the external auditor as relayed by the user, **not verified technical requirements**. No code, OAuth credentials, Gmail messages, or API interactions were involved in storing this summary.

A02 — DeepSeek — archiwum odpowiedzi

Źródło GitHub: https://github.com/luciferprosun/AIOA-spArkHAT/blob/feature/aioa-gmail-gateway/docs/audits/gmail_gateway/2026-10-20_DEEPSEEK_FULL_AUDIT.md

---

A02 — DeepSeek: AIOA spArkHAT Gmail API Gateway independent architecture, security and policy audit

Date: 2026-10-20 **Attribution:** DeepSeek, full 17-section response pasted by the owner **Source scope:** Single-owner, Linux-hosted Gmail integration for AIOA; one Google account, no multi-tenant SaaS. **Branch:** feature/aioa-gmail-gateway, starting from 81656e6... **Status:** RECEIVED_LONG_REPORT, fact check and actual repository integration review pending. No code or Gmail action authorized. **Archive fidelity:** All 17 sections, code models, matrix, scenarios, and source URLs transcribed in structured form; some formatting was normalized and some narrative condensed. The original unabridged wording remains in the source conversation.

---

1. Executive technical recommendation

Do NOT build a second authorization engine inside Gmail Gateway. The adapter is a client of existing AIOA Core, not a peer authority. Gateway cannot hold its own owner approval state, decide that a message is authorized, or treat tool calls as consent.

Use a desktop-native OAuth 2.0 Authorization Code with PKCE and loopback redirect on Linux. Refresh tokens belong in GNOME Keyring/libsecret or an AIOA-owned encrypted vault. MailProviderPort exposes typed owner-approved operations, no raw tokens. Only Gmail write capability: send an already-approved exact message. Read explicit selected threads as UNTRUSTED evidence.

Claimed rationale: quickest single-owner pilot without public web callbacks or separate mobile authority; temporary Testing mode with seven-day refresh token. Actual Google verification/CASA policy and production conditions require separate primary-source checking.

2. Existing AIOA vs new components

Component	DeepSeek proposal	Integration concern
CommunicationPort	Reuse/extend task and approval paths	Need actual repo inspection
MailProviderPort	Build narrow provider-neutral interface	Typed proposals/read/status
GmailAdapter	Build Gmail-specific MIME/REST/error mapping	No independent authorization

OAuthCredentialBroker	Build security-critical token custody	Tokens never model-visible
MessageProposal	Immutable, content addressed	Not an authorization
OwnerApprovalBinding	Reuse Core	Exact recipient/subject/body/attachment/thread/task hash
SendAuthorization	Short-lived single-use Core capability	Verify scope, expiry, binding
OutboxLedger	New, integrated with existing checkpoints	Durable record, not new scheduler
GmailThreadReader	New	Selected messages only
ReplyCorrelationVerifier	New	threadId, References/In-Reply-To, never From as identity
CommunicationAudit	Reuse Core audit + email receipts	Avoid parallel audit ledger of authority
HATStagingAdapter	Reuse HAT staging	Message provenance, untrusted flag
Android approval	Reuse operator controls	No additional signer

3. Google OAuth and Gmail API policy findings

3.1 Desktop OAuth

- Desktop Authorization Code + PKCE with loopback redirect 127.0.0.1 on ephemeral port.
- Official native app guidance: https://developers.google.com/identity/protocols/oauth2/native-app
- OOB manual copy/paste deprecated: https://developers.google.com/identity/protocols/oauth2/resources/oob-migration
- PKCE S256; generate high-entropy verifier/challenge. One observed report of PKCE exchange failing without a client secret is a testing signal, not a reason to abandon PKCE.
- Linux runs OAuth consent in its local system browser. Android remains only an AIOA operator UI and does not hold authorization code, refresh token, or client secret.
- Store refresh token in OS secret store, never .env, Git, logs or ordinary unencrypted database.
- Revocation endpoint: https://oauth2.googleapis.com/revoke. A revoked token, invalid_grant or changed account scope fails closed.
- External Testing mode: 100 test users, generally 7-day refresh token lifetime for Gmail scopes.
- Public sensitive/restricted apps may require OAuth verification. A security assessment applies under Google's restricted-scope processing conditions; exact timeline and cost claims in the model's report were estimates, not established facts.
- Multi-user requires separately designed user isolation and per-user OAuth.

3.2 Scope classifications claimed by DeepSeek

Scope	Claimed classification	Practical capabilities
https://www.googleapis.com/auth/gmail.send	Sensitive	Send messages only
https://www.googleapis.com/auth/gmail.readonly	Restricted	Read message/thread bodies
https://www.googleapis.com/auth/gmail.metadata	Restricted	Labels and headers, no body
https://www.googleapis.com/auth/gmail.compose	Restricted	Manage drafts and send
https://www.googleapis.com/auth/gmail.modify	Restricted	Broader read/update
https://www.googleapis.com/auth/gmail.labels	Non-sensitive	Labels only
https://mail.google.com/	Restricted	Broad/full mail access

MVP A: gmail.send only, local AIOA drafts, no Gmail drafts. MVP B: gmail.readonly only for messages/replies. MVP C: gmail.send + gmail.readonly. gmail.send alone cannot read the Sent folder or replies; gmail.compose is not needed for an in-AIOA draft.

3.3 Quotas, MIME, threading and send idempotency

- Claimed new project limits: 1,200,000 units/min per project and 6,000 units/min per user. messages.send 100 units, drafts.send 100, messages.get 20, messages.list 5, threads.get 40; newer daily billing threshold to verify.
- Pagination uses nextPageToken; do not assume one page is complete.
- Rate 429 and 5xx handling: truncated exponential backoff with jitter, but **NEVER automatically retry an ambiguous Gmail send**. Other read-only operations can use bounded retry.
- Gmail messages.send uses RFC 2822 MIME encoded base64url in the raw field. Attachments require multipart MIME.
- Threading may require threadId, consistent subject, References and In-Reply-To headers.
- Claimed Gmail could rewrite self-generated Message-ID, so treat it as a correlation hint, not an idempotency guarantee.
- No idempotency-key parameter for Gmail messages.send, so provider-level exactly-once cannot be guaranteed.
- UNKNOWN_SEND recovery: save intent and proposed Message-ID/hash before network call, query Sent/thread/readback where permitted, never blind retry. DeepSeek further proposed moving to CONFIRMED_NOT_SENT after five minutes with no match and permitting a new human-authorized attempt. **This inference is UNSAFE; an empty search is insufficient proof of non-delivery. Preserve UNKNOWN and require informed human decision; never certify negative delivery solely by absence in a search.**

3.4 Policy and privacy

- Google API Services User Data Policy / Limited Use: user-facing purpose, no resale/ads or disallowed data transfer. Privacy disclosures required under applicable distribution conditions.
- Google Workspace User Data/Developer Policy has approved productivity and email enhancement/reporting cases; app must describe actual use accurately.
- Raw Gmail data should not flow to third-party LLMs without appropriate consent/terms and explicit case-specific disclosure policy.

- Germany/EU: minimize personal data processing, encrypt stored content where needed, do not persist full mail into HAT by default; handle proper lawful basis, data retention/deletion and potential international data transfers. These are not final legal findings.

4. Three architecture alternatives

A — Desktop-native OAuth loopback + in-process narrow adapter (recommended). Pros: no public callback, no Android OAuth, simple integration with Core, best for a single owner. Cons: temporary Testing-mode expiry, desktop client secret isn't confidential, loopback service lifetime/binding requires correct handling.

B — Web OAuth client with public HTTPS callback. Pros: easier future multiuser deployment/central custody. Cons: public surface, state/callback/session/redirect security, more complexity and infrastructure. Not MVP.

C — Workspace service account with domain-wide delegation. Suitable only for controlled Workspace domain/admin and broad privileges; not appropriate for consumer single-owner Gmail. Reject.

5. Architecture dataflow

```
Android / ChatGPT presentation only
        |
        v
AIOA Core
  task state / Non-Zero / ServiceGuard / owner approval / STOP / replay
        |
        v
CommunicationPort / MailProviderPort (typed proposals; no tokens)
        |
        v
Gmail Gateway
  GmailAdapter ----- OAuthCredentialBroker (Keyring)
       |
  Durable OutboxLedger
       |
  ReplyCorrelationVerifier / CommunicationAudit / HATStagingAdapter
        |
        v
Gmail API (messages.send, messages.list, threads.get)

Reply direction:
Gmail API → selected thread reader → sanitized provenance envelope
  → untrusted evidence → Epistemic Control → HAT staging
  → separate human/policy review, NEVER automatic authority
```

6. Proposed module structure

```
aioa/
  communication/
    ports/
```

```
      mail_provider.py
      models.py
    gmail/
      __init__.py
      adapter.py
      auth/
        broker.py
        loopback.py
        keyring_store.py
      send/
        mime_builder.py
        outbox_ledger.py
        send_authorization.py
        unknown_recovery.py
      read/
        thread_reader.py
        reply_correlation.py
        sanitizer.py
      audit/
        communication_audit.py
      errors.py
tests/
  offline/
    fake_gmail.py
    test_outbox_ledger.py
    test_unknown_recovery.py
    test_reply_correlation.py
  integration/
    test_live_send.py
```

DeepSeek acknowledges this proposed top-level package may not match actual installed runtime layout and requires repo inspection before adoption.

7. Typed interfaces

DeepSeek proposed immutable dataclasses and a Protocol:

```python
from __future__ import annotations
from dataclasses import dataclass
from typing import Protocol, Sequence, Literal, Optional
from datetime import datetime

@dataclass(frozen=True)
class Recipient:
    address: str
    display_name: Optional[str] = None

@dataclass(frozen=True)
class AttachmentRef:
    filename: str
    mime_type: str
```

```
    sha256: str
    size_bytes: int

@dataclass(frozen=True)
class MessageProposal:
    proposal_id: str
    task_id: str
    thread_id: Optional[str]
    recipients: tuple[Recipient, ...]
    subject: str
    body_text: str
    body_html_sha256: Optional[str]
    attachments: tuple[AttachmentRef, ...] = ()
    in_reply_to: Optional[str] = None
    references: tuple[str, ...] = ()
    deterministic_message_id: str
    content_sha256: str
    created_at: datetime

@dataclass(frozen=True)
class SendAuthorization:
    authorization_id: str
    proposal_id: str
    task_id: str
    owner_approval_ref: str
    scope_granted: tuple[str, ...]
    expires_at: datetime
    single_use: bool = True

@dataclass(frozen=True)
class SendReceipt:
    proposal_id: str
    status: Literal["SENT", "UNKNOWN", "FAILED", "CONFIRMED_NOT_SENT"]
    provider_message_id: Optional[str]
    provider_thread_id: Optional[str]
    sent_at: Optional[datetime]
    reconciliation_hint: Optional[str]

@dataclass(frozen=True)
class ThreadMessage:
    message_id: str
    thread_id: str
    sender: str
    subject: str
    snippet: str
    body_text: Optional[str]
    received_at: datetime
    provenance: dict

class MailProviderPort(Protocol):
    async def validate_proposal(self, proposal: MessageProposal) -> list[str]: ...
    async def send_authorized(
```

```
        self, proposal: MessageProposal, auth: SendAuthorization
    ) -> SendReceipt: ...
    async def reconcile_send(
        self, proposal: MessageProposal, receipt: SendReceipt
    ) -> SendReceipt: ...
    async def get_thread(self, thread_id: str) -> Sequence[ThreadMessage]: ...
    async def list_selected_messages(
        self, query: str, max_results: int = 20
    ) -> Sequence[str]: ...
    async def get_message_metadata(self, message_id: str) -> dict: ...
```

Design requirements: model proposals cannot authorize; Core issues exact single-use token after actual owner action, full hash scope/recipient binding, broker hides refresh tokens, ledger before send, reconcile never sends, read functions return untrusted evidence. A mere Boolean single_use or approval_ref is not cryptographic proof; actual native Core authority mechanism required.

8. OAuth, token custody, privacy and human authority

- Store refresh tokens in keyring/libsecret or reviewed encrypted vault; access token in memory. Avoid storing any token fingerprints unnecessarily, especially across logs.
- Linux loopback binds 127.0.0.1 random port and shuts down on callback/short timeout; PKCE S256 and unpredictable state protect against interception/CSRF.
- Approvals bind exact recipient set, subject, body bytes/hash, attachments, task, proposal, scope and expiry; verify actual serialized MIME pre-send.
- Approval consumed exactly once durably; STOP before send, and ambiguous in-flight send must remain UNKNOWN, not STOPPED-with-guarantee.
- Revoked/expired refresh token requires explicit reauthorization; do not silently bypass.
- Logs contain limited nonsecret metadata; HAT staging stores source-backed summary not raw body; disclosure to external models needs explicit reviewed policy.
- Germany GDPR questions require actual data classification, deletion and legal basis review.

9. Durable send and UNKNOWN recovery

```
Model draft → Core validates and shows exact recipient/body
 → owner approves → Core issues exact scoped capability
 → adapter rechecks binding and STOP → ledger INTENT/SENDING durable
 → Gmail messages.send
   → success receipt/id: SENT
   → lost ACK/5xx/timeout: UNKNOWN_SEND, never repeat automatically
 → optional read-only reconciliation with gmail.readonly
 → if not proven: UNKNOWN_REQUIRES_HUMAN, not proof of non-delivery
 → any new send needs informed owner authorization with explicit duplicate risk
```

DeepSeek originally proposed treating absence after a five-minute search as CONFIRMED_NOT_SENT; this recommendation is archived but disputed. No native exactly-once guarantee exists. Even observed SENT status may not guarantee final recipient delivery.

10. Threat model: 14 negative cases

#	Threat	Defense/expected result

1	Model attempts send without approval	Core-bound authorizer denies; no Gmail call
2	Post-approval recipient/body tampering	Content hash mismatch, deny and audit
3	Email prompt injection	Treat as untrusted evidence; no instruction/authority
4	Spoofed collaborator reply	Thread/reference signals plus human identity confirmation
5	Gmail sends then network response lost	UNKNOWN_SEND; reconcile without resending
6	Restart/replay of task	Durable ledger idempotency prevents second call
7	OAuth token revoked	Fail closed, require owner reconnect
8	Android session compromised	No Gmail tokens on phone; separate Core authentication/approval
9	LLM data exfiltration	Redacted context, model egress policy and owner disclosure
10	HTML/attachment hostile content	Sanitize/quarantine, type/size/source caps
11	Two agents racing same thread	Single-writer/serialized ledger, conflict result
12	Approval reused on another message	Fingerprint/task/expiry/single-use rejection
13	Gmail quota exceeded	Bounded backoff for safe READ, not ambiguous send retry
14	STOP during send	Check before network call; if in-flight return UNKNOWN

For high-impact statements or purported collaborator consent, sender name, From header and even mail authentication results are indicators, NOT identity or broad legal permission.

11. Multi-model orchestration

AIOA models only see a typed propose-message tool, never credential or raw HTTP client. One model may draft, another critic may review, but neither approves or sends. Provider hops and context resets do not elevate/preserve authority. AIOA Core revalidates task, exact proposal, scope, owner decision, expiry and STOP. Store minimal audit envelope: task/proposal/authorization IDs, recipient hash, content hash, provider ID/thread ID and result; no raw personal mail body.

12. Roadmap / estimates for low-RAM Linux PC

Phase	Features	Tests / gates	Estimate
G0	Repo integration review, typed mail contracts, threat model	No network, no credentials; tests fail closed	1–2 days

G1	Fake Gmail provider, durable ledger, UNKNOWN/reply correlation	Offline replay, spoof, injection, duplicated tasks	3–5 days
G2	Linux Desktop OAuth, PKCE loopback and keyring	Real test-account consent, refresh/revoke/expiry; owner action	2–3 days
G3	Exact single-owner-approved Gmail send and readback	First send to test recipient; content binding and LOST-ACK simulation	3–5 days
G4	AIOA provider orchestration + Epistemic Control + HAT staging	Model A drafts, B critiques, owner approves, reply is untrusted	5–8 days
G5	Android operator view through existing Core	No phone Gmail token, independent mobile signer or public endpoint	3–5 days

Auditor's original total estimate: ~17–28 working days for G0–G5, with ~9–15 working days to initial live send. This is an estimate not a schedule guarantee.

13. Chromium future boundary — no implementation

Keep provider-independent CommunicationPort/CapabilityGrant/EvidenceEnvelope/ApprovalBinding. Future managed-browser/Chromium implementation may use these interfaces under the same Core authorization, but do not implement browser, Chrome OAuth or public REST listener in Gmail MVP.

14. GOLD72 isolation

Gmail G1 fake conversations can run in isolated offline/SHADOW mode without modifying sealed GOLD72 source. Live pilot uses separate runtime/credentials/outbox/evidence and its own limits. HAT records from email are staged as untrusted. A separate Google Cloud project recommended. Gmail delivery should NOT become necessary for unrelated GOLD72 reliability PASS.

15. Ten architectural risks

1. Second authorization engine in adapter.
2. False exactly-once claim.
3. Restricted scope verification/CASA underestimation.
4. Prompt injection in email.
5. OAuth token leakage.
6. Stale/reusable approval.
7. Seven-day Testing refresh expiration.
8. Android independent approval/token.
9. Message-ID rewrite and reconciliation ambiguity.
10. Cross-contamination with Linux Sprint33/34 or GOLD72.

16. Errors/unsafe assumptions identified by DeepSeek

1. gmail.send is Sensitive, not Restricted.
2. gmail.compose is Restricted, so do not use Gmail drafts for MVP.
3. Gmail lacks native exactly-once messages.send.
4. From does not establish sender identity.
5. Android should not hold Gmail OAuth credentials for this Linux custody design.
6. Testing mode is a temporary piloting solution, not a durable production strategy.

7. Repo features described in initial prompt are not automatically production ready.
8. A public Web OAuth callback is unnecessary for a local single-owner desktop test.

17. Final implementation recommendation and acceptance gates

DeepSeek chooses Desktop Loopback OAuth + PKCE, MailProviderPort, existing AIOA Core sole authority, exact content-bound single-use send authorization, durable OutboxLedger, honest UNKNOWN_SEND, and untrusted HAT staging. First build G0/G1 offline with full negative tests. Then G2/G3 on owner-approved *test account* using gmail.send first, adding gmail.readonly for selected replies only after policy/consent review. No Chromium, external API access or GOLD72 runtime mutation.

Required before connecting owner account:

1. Offline G0/G1 PASS.
2. Explicit owner consent, preferably dedicated test Gmail first.
3. Dedicated GCP project/test-user config.
4. Desktop loopback 127.0.0.1 + PKCE S256.
5. Keyring/vault token custody and redacted logs.
6. Exact first-send owner review.
7. STOP tested before first live send.
8. Durable ledger + UNKNOWN negative tests.
9. Android UI only; no Android OAuth client.
10. No shared mutable GOLD72/Sprint33 runtime.

Blocked: broad scopes, unapproved outgoing messages, silent resends, auto HAT trusted promotion, email data egress without permission, publishing/CASA actions, remote bridge/Chromium or production LIVE claims.

Official links supplied by DeepSeek

- Desktop native OAuth: https://developers.google.com/identity/protocols/oauth2/native-app
- OOB migration: https://developers.google.com/identity/protocols/oauth2/resources/oob-migration
- Scope table: https://developers.google.com/workspace/gmail/api/auth/scopes
- Quotas: https://developers.google.com/workspace/gmail/api/reference/quota
- Workspace data/developer policy: https://developers.google.com/workspace/workspace-api-user-data-developer-policy
- Limited Use: https://developers.google.com/terms/api-services-user-data-policy
- users.messages.send: https://developers.google.com/workspace/gmail/api/reference/rest/v1/users.messages/send
- Threading: https://developers.google.com/workspace/gmail/api/guides/threads
- Push: https://developers.google.com/workspace/gmail/api/guides/push

---

Archive integrity statement: This archive is a structured, substantive transcription of the supplied 17-section report with narrative compression and formatting cleanup. It does not present itself as byte-for-byte original. The exact chat text remains the primary user-supplied evidence. Google factual assertions and any code contract require separate official-documentation and AIOA source verification.

A03 — Claude Sonnet — archiwum odpowiedzi

Źródło GitHub: https://github.com/luciferprosun/AIOA-spArkHAT/blob/feature/aioa-gmail-gateway/docs/audits/gmail_gateway/2026-10-20_SONNET_AUDIT_RECEIVED.md

---

A03 — Sonnet independent Gmail Gateway audit (received answer)

Date: 2026-10-20 **Author:** Claude Sonnet, per user attribution **Completeness:** The provided answer is preserved below; the model offered but did not supply a full 16-section report. **Status:** UNVERIFIED, research only; no code or live access approved.

I researched the scope classifications and answer your final question below. I haven't inspected the repo, so anything about existing AIOA modules is an assumption. The full 16-section report would be long, so I can put it in a doc if you want it.

Recommended architecture Build it as an in-process Python adapter behind the existing Core, not a separate service. Use no REST endpoint and no second authority plane. You need about four new pieces; the rest of your 12 proposed components are probably existing Core features under new names (pending repo check):

- MailProviderPort: a narrow interface with create_draft, send_exact, read_thread and reconcile.
- GmailAdapter: the only module that touches tokens. Models never see credentials or the Gmail client.
- OutboxLedger: SQLite with a state machine of PROPOSED → APPROVED → SENDING → SENT | UNKNOWN_SEND | FAILED.
- ReplyCorrelator: matches replies by thread ID, In-Reply-To and your request ID.

Approval should bind to a hash over {task_id, recipient, subject, body, attachments, scope, expiry}. The adapter recomputes that hash at send time and refuses any mismatch. Core should reuse its existing approval, STOP and replay logic.

Scope classification Google's scopes page lists gmail.send as sensitive and gmail.readonly, gmail.compose and gmail.metadata as restricted. The snippet I retrieved was garbled around gmail.modify and mail.google.com. Third-party guides classify both as restricted and gmail.labels as non-sensitive, but verify those directly. Restricted scopes bring verification and a security assessment if you distribute the app. For a single-owner app in Testing mode, the practical issues are a warning screen and a 7-day refresh-token expiry (from memory; verify).

Minimum scopes

- **MVP A (draft and send):** gmail.send only. It sends but cannot create drafts, so drafts live in AIOA, which is the better design anyway.
- **MVP B (read and summarize):** gmail.readonly, which is restricted. gmail.metadata can't read bodies and, as far as I know, can't search them. Please verify that.
- **MVP C (send and monitor):** gmail.send plus gmail.readonly.

A flaw in the current proposal gmail.send alone cannot read the Sent folder, so it can't reconcile UNKNOWN_SEND. Gmail has no native idempotency for messages.send, so the safe design is:

1. Persist SENDING before the call.
2. On a lost response, move to UNKNOWN_SEND and block all retries.

3. Reconcile either by the owner checking Sent, or by a gmail.readonly search for the request ID embedded in the message.

With send-only scope, reconciliation is therefore manual. Whether Gmail preserves a client-supplied Message-ID is not something I verified, so test it in G2.

OAuth

- Use a Desktop client with the loopback redirect and PKCE.
- The old out-of-band flow is deprecated.
- Authorization happens on the Linux machine; Android only triggers it. No tokens ever cross to the phone.
- Store the refresh token in the OS keyring or a mode-0600 file outside the repo, and revoke it via Google's revoke endpoint on STOP.
- Treat invalid_grant as a hard failure that needs the owner.

Fastest route to the first real email

1. **G0–G1 (about 2–3 days):** build the simulator, ledger, hash binding and negative tests offline.
2. **G2 (about 1 day):** connect a dedicated test Gmail account with gmail.send only.
3. **G3:** send one owner-approved message to yourself first, then to Natalia.

Reply handling comes after that. Until then the owner pastes her reply as untrusted evidence. A pasted reply carries no restricted scope, no sender-identity claims and no verification burden.

Conditions for connecting your account

- The offline suite is green, including the double-send, stale-approval and STOP-mid-call tests.
- You use a test account first, not your main one.
- GOLD72 is isolated (separate runtime, credentials and evidence).
- You've read the exact consent screen.
- No message contents go into logs, prompts or the repo.

Blocked until further authorization

- Any gmail.readonly, gmail.modify or mail.google.com scope.
- Auto-send and auto-retry after UNKNOWN.
- Sending any real email before G3 approval.
- Forwarding email bodies to external model providers. Google's Limited Use rules and GDPR both apply, and Natalia's address and her reply are third-party personal data.
- Chromium work.
- An email-derived claim that could authorize execution or change policy.

Further mistakes in the brief

- Treating sender identity as provable. Gmail's Authentication-Results header (SPF/DKIM/DMARC) is a signal, not proof.
- Reading a reply's "consent" as broader than the message itself.

I'd verify the quota numbers, the Testing-mode token lifetime, Message-ID handling and gmail.metadata search limits against Google's docs before G2.

External audit archived as received; Google policy claims need separate primary-source checking.

A04 — Gemini — archiwum odpowiedzi

Źródło GitHub: https://github.com/luciferprosun/AIOA-spArkHAT/blob/feature/aioa-gmail-gateway/docs/audits/gmail_gateway/2026-10-20_GEMINI_AUDIT_RECEIVED.md

---

A04 — Gemini: AIOA spArkHAT Gmail API Gateway independent audit

Audit delivered by: Gemini (per owner attribution)
Date: 2026-10-20
Evidence: Full long response pasted by owner. Section order and core content preserved below in readable Markdown; formatting normalized.
Verification: External AI output — NOT independently accepted. In particular, Google's official scope table contradicts some of Gemini's classifications; see audit registry comparison.

Executive Technical Recommendation

To integrate Google Gmail API into **AIOA spArkHAT** without violating AIOA's core security invariants (**Provider is not authority**, **ZERO_WRITE by default**, **Human approval by default**, and **One Core**), we recommend a **Decoupled Outbox-Ledger Architecture with Mandatory Hash-Bound State Machine Seals**.

Core Decision Highlights

1. **Zero-Trust Outbox Ledger:** AI models NEVER issue API calls or handle raw OAuth tokens. Models create a strictly typed, immutable MessageProposal inside the AIOA Core. Core enforces a cryptographic ApprovalBinding containing SHA-256 hashes of the exact recipient, subject, body, attachment hashes, and task ID.
2. **Explicit Human Authority via Loopback OAuth:** For a single-owner Linux environment (Ubuntu/Mint), use a Desktop App (Native/Loopback) OAuth client configuration using PKCE (RFC 7636). Refresh tokens are stored encrypted via OS-level secret storage (SecretService/keyring with AES-256-GCM backed by a passphrase/hardware TPM/keychain), completely inaccessible to model context or agent processes.
3. **Strict Scope Minimization:**
   - **MVP A (Draft & Send):** gmail.send (**Gemini claims Restricted**) or gmail.compose (**Gemini claims Sensitive**) + gmail.send.
   - **MVP B & C (Full Bidirectional Exchange):** gmail.send + gmail.readonly.
   - **Banned Scopes:** mail.google.com (full access—excessive and dangerous) and gmail.modify/gmail.labels (unnecessary for simple communication).
4. **Deterministic UNKNOWN_SEND Protocol:** Gmail API's users.messages.send does NOT provide native idempotency keys. Network drops/5xx can cause duplicate sends if blindly retried. Gateway enforces pre-send draft creation or cryptographic Client-Header tracking. If HTTP response is lost, move to UNKNOWN_SEND and freeze automatic retries until reconciliation checks (users.messages.list by RFC 5322 Message-ID) verify status.

1. Existing AIOA Baseline vs. New Gateway Requirements

Based on repository inspection and architectural alignment with AIOA-spArkHAT:

Architectural responsibility	Existing AIOA system capabilities (claimed)	Required Gmail Gateway extensions
Authority & approvals	Human-controlled authority, ServiceGuard, STOP controls	OwnerApprovalBinding with payload hash seal
Model operations	MCP Bridge / MCP Commander, Bounded Execution	Typed GmailToolCall schema (proposals only)
Data safety	Non-Zero safety, ZERO_WRITE default	Outbox ledger with DRAFT_ONLY default posture
Epistemic integrity	Epistemic Control, independent criticism	ReplyCorrelationVerifier & untrusted evidence tagging
State & persistence	Checkpoints, receipts, persistent task state	OutboxLedger table + UNKNOWN_SEND reconciliation
Knowledge capture	HAT knowledge and memory management	HATStagingAdapter for extracted email evidence

2. Google OAuth 2.0 & Gmail API Policy Research

OAuth 2.0 Architecture

- **Client type:** Desktop Application: https://developers.google.com/identity/protocols/oauth2/native-app
- **Flow:** Authorization Code + PKCE (RFC 7636). Linux app launches loopback web server, such as 127.0.0.1:AUTO_PORT/oauth2callback.
- **Deprecated/unsafe:** Out-of-band manual copy/paste (urn:ietf:wg:oauth:2.0:oob) deprecated and blocked; embedded WebViews blocked.
- **Android/Linux:** Android acts only as remote client UI; no Google secrets or refresh token on Android. Linux AIOA Core initiates authorization or retrieves locally stored credentials.
- **Custody:** OS secret stores with encrypted-at-rest refresh tokens and 0600 file permissions.
- **External Testing mode:** Explicitly allow up to 100 test users; Gemini claims refresh tokens expire after 7 days if not continuously refreshed. The final clause requires correction against Google docs.

Gmail scopes and policy verification classifications

The following table preserves Gemini's claims and is NOT validated: Google's current official page classifies gmail.send Sensitive; gmail.compose and gmail.metadata Restricted, contrary to Gemini.

Scope	Gemini's classification	Use
gmail.send	Restricted (incorrect)	MVP A/C send
gmail.readonly	Restricted	Read/search
gmail.compose	Sensitive (incorrect)	Draft operations
gmail.metadata	Sensitive (incorrect)	Headers/labels
gmail.modify	Restricted	Not recommended

mail.google.com	Restricted	Broad full access, banned by proposed policy

Gemini asserted restricted scopes require Limited Use compliance and for public apps a CASA security assessment; one-owner test app could use Testing mode. Actual applicability requires evaluating Google policy, data storage/transmission and user distribution context.

3. Recommended Architecture Diagram & Data Flow

```
AI AGENT LAYER
  Multi-Model LLM / Epistemic Critic / Prompt Context (NO TOKENS)
                   |
                   v Propose + critique
AIOA CORE GATEWAY
  MessageProposal Engine
                   |
  Cryptographic Approval Binding
  SHA-256(recipient, subject, body, attachments, task ID)
                   |
  HUMAN OWNER UI: exact text + digest → APPROVE/REJECT
                   |
  ServiceGuard + SendAuthorization
                   |
  Gmail Adapter + Outbox Ledger + Token Custody
                   |
                   v HTTPS
Google Gmail API: users.messages.send / users.threads.get / users.messages.list
```

Models can propose drafts. Core computes/revalidates exact hashes and obtains explicit human authorization. The adapter enforces ledger state and calls Google.

4. Proposed Repository File & Module Structure

New self-contained proposed module under aioa/gateways/gmail:

```
aioa/
  gateways/
    gmail/
      __init__.py
      contracts.py         # typed dataclasses and enum definitions
      exceptions.py
      oauth_broker.py      # PKCE and Linux keyring
      approval.py          # hash and owner approval binding
      outbox_ledger.py
      adapter.py           # Gmail REST/MIME send and read
      thread_reader.py
      hat_staging.py
tests/
  gateways/
    gmail/
      test_approval_binding.py
```

```
      test_outbox_ledger_state.py
      test_idempotency_recovery.py
      test_security_threats.py
```

5. Typed Contracts & Minimal Interface Definitions

Gemini proposed Pydantic schema definitions:

```python
import enum
import hashlib
from datetime import datetime
from typing import Dict, List, Optional
from pydantic import BaseModel, EmailStr, Field

class OutboxState(str, enum.Enum):
    PROPOSED = "PROPOSED"
    APPROVED = "APPROVED"
    REJECTED = "REJECTED"
    SENDING = "SENDING"
    SENT = "SENT"
    UNKNOWN_SEND = "UNKNOWN_SEND"
    FAILED = "FAILED"
    REVOKED = "REVOKED"

class MessageProposal(BaseModel):
    proposal_id: str
    task_id: str
    sender: EmailStr
    recipient: EmailStr
    subject: str
    body_text: str
    body_html: Optional[str] = None
    custom_message_id: str = Field(description="RFC 5322 Unique Message-ID for
tracking")
    attachment_hashes: List[str] = Field(default_factory=list)

    def compute_payload_hash(self) -> str:
        """Computes deterministic SHA-256 seal over proposal content."""
        hasher = hashlib.sha256()
        hasher.update(self.task_id.encode('utf-8'))
        hasher.update(self.recipient.lower().encode('utf-8'))
        hasher.update(self.subject.encode('utf-8'))
        hasher.update(self.body_text.encode('utf-8'))
        for att in sorted(self.attachment_hashes):
            hasher.update(att.encode('utf-8'))
        return hasher.hexdigest()

class OwnerApprovalBinding(BaseModel):
    approval_id: str
    proposal_id: str
    payload_hash: str
```

```
    approved_by_owner: bool
    timestamp: datetime
    owner_signature: str  # HMAC or signed token from owner UI

class Receipt(BaseModel):
    proposal_id: str
    gmail_message_id: Optional[str]
    gmail_thread_id: Optional[str]
    state: OutboxState
    error_details: Optional[str] = None
    executed_at: datetime
```

Cross-audit concern: The proposed concatenated-string SHA-256 omits sender, body_html and other material and lacks unambiguous canonical serialization. This has NOT been adopted.

6. Delivery, Idempotency and UNKNOWN_SEND Recovery

Gemini proposes:

```
INITIATE SEND
  → Create deterministic RFC 5322 Message-ID header
  → Durable ledger SENDING
  → POST users.messages.send
      ├─ HTTP 200 → SENT
      └─ Network failure / 5xx → UNKNOWN_SEND
           → query users.messages.list(rfc822msgid:...)
               ├─ FOUND → SENT, no retry
               └─ NOT FOUND → FAILED / re-approval
```

Details:

1. Inject Message-ID: AIOA-TASKID-PROPOSALID@domain.com into raw MIME.
2. Persist SENDING on disk before API call.
3. On loss of response transition to UNKNOWN_SEND and NEVER automatically retry.
4. Reconcile by Gmail messages.list query; Gemini claims nonempty result means delivered and empty result means never delivered, then FAILED with re-approval.

Critical audit caveat: A negative search result does NOT prove non-delivery; UNKNOWN must remain until sufficient independent evidence/human judgment. Gmail does not guarantee exactly-once, and a matching Message-ID is not guaranteed unique API idempotency.

7. Security Threat Model & Mitigation Matrix

Scenario	Risk	Mitigation	Failure/recovery
Unauthorized send by model	Critical	ServiceGuard & hash-bound owner approval	Reject, log
Recipient/body modification	Critical	Verify exact payload hash	Block, alert
Indirect prompt injection	High	Untrusted external evidence + Epistemic Control	Isolate, no effect

Forged collaborator reply	High	SPF/DKIM metadata signals, no authority	Informational only
Stale approval reuse	Medium	Proposal ID, single use, 15-minute expiry	Reject
Credential leakage in prompts/logs	Critical	OS keyring, logging sanitization	Secret-pattern tests

8. Implementation Roadmap

- **G0 — Contracts & threat model (3 days):** contracts, exceptions, Pydantic schemas, tests for hash seals.
- **G1 — Offline Gmail simulator & outbox ledger (4 days):** fake API simulates network loss/UNKNOWN; durable ledger with file locking.
- **G2 — Real OAuth & token custody (3 days):** Desktop PKCE loopback, Linux keyring; authorized test account.
- **G3 — Controlled pilot (4 days):** approved send and reply readback; UNKNOWN recovery.
- **G4 — Multi-model orchestration (5 days):** Gemini drafting, Claude critique, Epistemic Control and HAT staging.
- **G5 — Android approval UI integration (4 days):** display and signing via existing Core.

9. Future Architecture Boundaries

The proposed CommunicationPort and MailProviderPort abstractions allow later browser-based webmail or Chromium drivers beneath the same AIOA Core/ServiceGuard/approval boundary. No Chromium implementation now.

GOLD72 remains an isolated runtime; Gmail gateway optional OFFLINE_SIMULATOR mode with synthetic messages only. No external network dependence in qualification.

10. Top 10 Architectural Risks

1. Gmail messages.send has no native idempotency; duplicate send on ambiguous failure.
2. Testing-mode refresh token lifetime misinterpreted.
3. Prompt injection in inbound mail.
4. Approval payload displayed by UI may not match exact raw MIME bytes.
5. OAuth token leakage in HTTP traces.
6. Quota exhaustion/HTTP 429; Gmail account daily send limits.
7. Multiple models acting on one thread.
8. Loopback port unavailable in headless Linux.
9. Accidental mail.google.com/ overly broad scope.
10. Interrupted durable state write during power loss.

11. Final Recommendation

Adopt Decoupled Outbox-Ledger Architecture with cryptographic hash seals and RFC 5322 Message-ID reconciliation.

Proposed Gmail scopes:

- https://www.googleapis.com/auth/gmail.send

- https://www.googleapis.com/auth/gmail.readonly

Quickest route: G0 contracts and G1 simulator offline, then G2 Google Cloud Desktop OAuth client in Testing, then G3 one manually approved send.

Blocked: no auto-send without owner, no mail.google.com, no untrusted email command execution.

Preservation note: Some HTML entities and table layout in the original pasted answer were normalized. Claims, technical direction, schema, and risks are recorded for independent comparison, not as verified design facts.

A05 — Kimi — archiwum odpowiedzi

Źródło GitHub: https://github.com/luciferprosun/AIOA-spArkHAT/blob/feature/aioa-gmail-gateway/docs/audits/gmail_gateway/2026-10-20_KIMI_AUDIT_RECEIVED.md

---

A05 — Kimi independent Gmail Gateway audit (received summary)

Date: 2026-10-20 **Author:** Kimi, per user attribution **Completeness:** Only the supplied summary; the model says a longer PDF-ready Markdown exists but did not provide the document. **Verification:** Unverified external model findings; NOT implementation authority.

I'll approach this as an independent audit: first verify current Google OAuth/Gmail policy facts against official documentation, then inspect the actual repository before making any claims about what exists.

I got the quota and scope facts (with some important 2026 changes to verify). The direct URL fetches were rejected, so I'll use targeted searches against official documentation for the remaining policy facts.

The README confirms the real repository layout. Let me inspect the key architecture docs and runtime modules to ground the audit in actual code structure.

I have enough verified policy facts and repository grounding. Now I'll write the full audit report to a PDF-ready Markdown file.

The full audit is complete and saved as a PDF-ready Markdown report. Summary of the load-bearing findings:

Verified Google policy facts (do not re-derive from memory):

- **Scopes:** gmail.send is *Sensitive* (send-only, no read, no security assessment); gmail.labels is non-sensitive; gmail.readonly, gmail.compose, gmail.metadata, gmail.modify, and mail.google.com/ are all *Restricted* — restricted scopes trigger verification plus an annual security assessment for production, and the Limited Use policy fully applies. Workspace policy confirms any Gmail scope that reads/creates/modifies mail is restricted.
- **Quotas changed 2026-05-01:** new projects get 1.2M units/min per project, 6,000 units/min per user, 80M-unit daily billing threshold; messages.send costs 100 units, threads.get 40, messages.get 20, history.list 2. Confirm your project's actual values in the Cloud Console, since older projects keep legacy quotas.
- **OAuth:** Desktop/installed client with loopback + PKCE is the correct flow; the OOB copy-paste flow was blocked for new clients from Feb 2022. Testing mode = 100 test users and **refresh tokens expiring in 7 days** — plan weekly re-auth or publish the app, and never "fix" expiry by re-consenting in a loop (50-token silent revocation cliff).
- **Idempotency: Gmail messages.send has none.** Idempotency keys are a provider-specific feature that some modern email APIs added explicitly — Gmail is not among them. The entire reliability design therefore hinges on the self-generated RFC 822 Message-ID + UNKNOWN_SEND + human-gated reconciliation in the report's §8.

Core architectural verdicts:

1. Your proposed component list is 60% rename-and-reuse; only the adapter, credential broker, outbox ledger, and reply correlator are genuinely new. The repo's existing

MCP_APPROVAL_TOKEN_V1_FIXTURE, ServiceGuard, receipt/checkpoint chain, and FORBIDDEN_MEMORY_FLOWS discipline absorb most of it — but that must be confirmed by a half-day spike against a local checkout (all repo claims in the report are flagged as structural, not line-level).

2. The "tool sidecar" and "Gmail-drafts staging" architectures are rejected; the recommended design is an in-Core runtime/gateway/ module behind a ChannelPort, executed as a ServiceGuard effect.
3. Build the UNKNOWN_SEND state machine and the 14-case negative test suite **before any credential exists** — fastest honest route to the first real email is 1 week (G0→G1→G2, gmail.send only).
4. The published-but-unverified-app token-lifetime behavior for restricted scopes is a flagged gray zone to confirm at G2.

Record limitations: The Kimi full report was NOT attached in the user's message. Claims of repo inspection and up-to-date documentation were made by Kimi and have not been independently authenticated. The model's phrase about *50-token* silent revocation differs from Google's separately published limits and must be checked rather than treated as fact.